

Experience Report of Teaching Agile Collaboration and Values

Agile Software Development in Large Student Teams

Martin Kropp

University of Applied Sciences
Northwestern Switzerland
Windisch, Switzerland
martin.kropp@fhnw.ch

Andreas Meier

Institute of Information Technology
Zurich University of Applied Sciences
Winterthur, Switzerland
meea@zhaw.ch

Gerald Perellano

Institute of Information Technology
Zurich University of Applied Sciences
Winterthur, Switzerland
pero@zhaw.ch

Abstract— Communication and collaboration are central skills for agile development. Trust, openness, transparency and equality are core agile values and there is a shortage of software developers with these skills. How can we teach software engineering so that students get these skills together with the needed technical competences? This paper presents and discusses the setup of an innovative agile educational student project. The didactical concept is based on results from several industry studies, which give insight into the required competencies of agile software developers.

Keywords— agile; collaboration; values; software development; education

I. INTRODUCTION

Recent studies show that agile methodologies have become mainstream in the software industry [1], [2]. At the same time these and other studies show that especially *collaborative practices*, like Pair Programming, Daily Stand-up meetings and Retrospectives are rarely applied by inexperienced teams [2], [3], [6]. Consequently, reports about dysfunctional teams and disastrous projects in inexperienced agile teams are, unfortunately, quite common [4], [5].

So what do experienced and successful teams do differently? The mentioned studies show that the usage of collaborative practices grows gradually over time. In [6] the authors conclude that “experience leads to collaboration”. In other words, only mature teams apply most collaboration practices. Mature teams also apply many more *technical practices* like Automatic Testing, Continuous Integration, Test-Driven Development, Refactoring or Clean Code. In [3] the authors found that successful teams also highly respect the *agile values*, like *Commitment, Focus, Openness, Respect and Courage* [15], or *Simplicity, Communication, Courage, Respect and Feedback* [16]. What seems to be most important is that the team behavior and the culture of the organization are aligned to the agile values [6].

This lack of usage of collaboration practices in inexperienced organizations seems to suggest, that there is a high demand for well-educated software engineers in the agile domain who have the necessary collaboration and

communication skills and who can live the agile values. Therefore, it is important, not only to give students a good understanding of what to expect when working in self-organized, agile software development teams, but also that the agile values are discussed with and experienced by the students during a tangible student project.

This paper presents the setup of a “realistic” agile student project in which the students develop an application in large, self-organized, teams. In these projects, the students have the opportunity to experience and experiment with agile collaboration practices and values.

Based on the results of the recent studies, the authors, who have been teaching software engineering and agile methods for more than a decade, started developing a new software engineering module. It especially addresses agile methodologies with a much higher emphasis on teaching collaboration practices and agile values. In this paper, the authors present the agile student project in the fourth semester of the Computer Science studies at the Zurich University of Applied Sciences. The agile student project is an important part of the new software engineering module. The scope of the student project is equivalent to four ECTS credit points (European Credit Transfer and Accumulation System). One credit point is equivalent to approximately 30 hours of studying; the total was therefore 120 hours.

The rest of the paper is structured as follows: Section two discusses related work. Section three presents the didactical concept and its impact on the new module. Section four describes the setup of the agile student project and presents how collaboration and agile values are taught and practiced in the classroom. Section five evaluates and discusses our observations, followed by final conclusions.

II. RELATED WORK

Teaching agile software development and software processes has also reached academia in the meantime. There are many courses offered at universities all over the world.

Mahnic [7] provides a thorough overview of approaches of teaching agile software development. He shows that teaching Scrum in practical work is the most wide spread approach. He also describes the various approaches of

assigning the different Scrum roles with its pros and cons. He especially argues that assigning the Product Owner role to a domain expert, gives a more realistic simulation with real customer collaboration.

Schroeder et al. [8] report on the setup of two software development labs specifically designed so that the social interaction “how to deal with collaborative software development processes and their need for self-organization, motivation, and work coordination”, is taken into conscious consideration. Scharf and Koch [9] present the design of a Scrum based undergraduate software engineering course, which is based on role-playing. To ensure that their students understand “agile” thoroughly, they designed the complete course as realistically as possible. For instance, they make research assistants play customers and a student play the Scrum master. Stettina et al. [10] discuss their experiences with teaching software engineering practices and their continuous improvement. They emphasize the importance of an intensive coaching routine based on agile practices.

In [11] Maurer and Anslow report about their experiences teaching a group based agile software development project course. They report that students enjoy the interactive exercises and the self-organized teams, but also that it is challenging to get the right customer involvement. In [12] Rico and Sayani present their results after teaching student teams agile methods in their final year. They report that those teams who found the optimal balance of customer collaboration, technical programming and use of agile methods were more productive. Little work has been done specifically addressing the issue of teaching collaboration in the classroom. In [13] Chao describes how he used wikis in a student project setup and reports that students liked this medium a lot and is therefore planning to continue using wiki in future projects. In [19] Kropp et al. describe the importance of advanced collaboration skills in agile development and make suggestions on how to integrate them in undergraduate courses

III. DIDACTICAL CONCEPT

The overall educational objective of our student project is: “The students know by experience, what it feels like to work in a self-organized, functional agile software development team. And they know what distinguishes functional teams from dysfunctional ones.”

In order to successfully develop software in an agile team, the students need many different competencies. The setup of the student project is formed with help of the agile competence pyramid in Fig. 1. The pyramid divides the needed agile competencies in the three categories: Technical practices, collaboration practices and agile values. The didactical concept of the student project strives to integrate these competencies by carefully setting up a realistic environment in the classroom.

A. Technical Practices

The students have been taught technical practices like automatic testing, refactoring, continuous integration or clean code in previous lectures. In the student project, the focus is on applying and refining them in practice. For

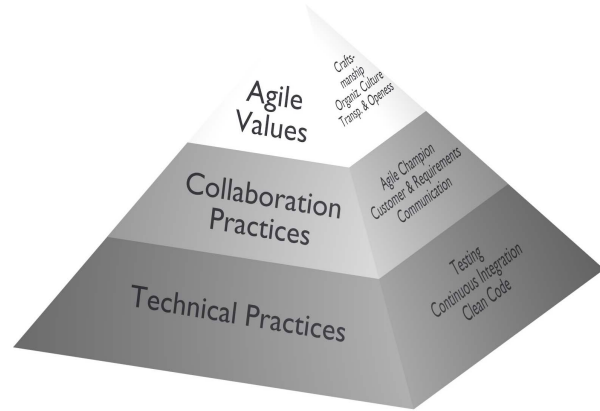


Figure 1. Agile Competence Pyramid [3]

instance, clean code and craftsmanship [14] are discussed frequently and practiced thoroughly.

B. Collaboration Practices

One of the key differences between agile and plan-driven software development methodologies is the emphasis on collaboration, communication and feedback. Collaboration practices like iteration or release planning, retrospectives, daily standup, pair programming or code review are encouraged and facilitated by the coach.

C. Agile Values

The agile values are on top of the pyramid because they are of utmost importance in agile software development. One of the key-ideas of the student project is to create a realistic environment in which the students can experience and safely experiment with the agile values. The authors believe that just teaching the agile values is insufficient; they have to be experienced in practice. It is the responsibility of the coach to create an appropriate ecosystem.

D. From Teaching to Coaching

Since agile teams are self-organized, the lecturer takes on the role of a coach and agile champion. The coach helps the team to continuously improve the process and to remove impediments, i.e. the coach helps to create functional agile teams. Last but not least, the coach is responsible that the agile values are practiced and respected.

E. Student Team Size

Next to the competencies, the size and constitution of the student teams are vital. Optimal development team size is small enough to remain agile and large enough to complete significant work within an iteration or Sprint. In Scrum the recommendation is seven, plus or minus two [15]. Fewer than five team members decrease the necessity for interaction and collaboration. Smaller teams might also encounter skill constraints. More than nine members require too much coordination. Large development teams generate too much complexity for an empirical process, like Scrum, to manage.

What is an optimal size of student teams? The authors had best results when the team size was at the upper limit. In

smaller student teams there is a risk that the students do not realize the necessity of proper project management. Much too often, project management is seen as an overhead and, in worst case, is only applied for the lecturer's sake. In properly sized teams, project management is crucial for success. Another benefit of larger teams is, that the students get more work done, which increases the chance of implementing complete results in more realistic software applications. In our experience optimal student team size is between seven and nine members.

IV. AGILE STUDENT PROJECT

Table 1 shows the whole semester schedule for the student project. The following sections will describe the important concepts.

A. Scrum

We chose Scrum [15] because it covers most of the collaboration practices and because of the good experiences we had previously made in industry and research projects. Further, Scrum is the most widely used agile methodology [1] in Switzerland. For the student project, Scrum was carefully adapted to fit the special requirements in the classroom as described in the following paragraphs.

B. Finding a Project Vision

Before the start of the semester, we issued a *call for projects*. Students, faculty members of the Computer Science department and industry partners could submit project ideas. At the kick-off meeting, each valid proposal was presented to the students by the submitter in a two-minute presentation. After that, the students could ask questions to clarify the key issues of the project. Finally, the students voted for the best projects and the teams were formed. After approximately three hours, the project visions and team formation was finished.

TABLE I. STUDENT PROJECT

Before start of semester	Students and faculty are informed about the project module and asked to send in project ideas.
Week 1 (Kick-off meeting)	Kick-off Meeting. Presentation and election of best-suited project ideas. Composition of student teams
Week 2	Introduction of the IT-infrastructure. Exploring the infrastructure. Teams are trying out frameworks and tools for their project. User Stories are written and estimated. First version of Product Backlog is written
Week 3 (Sprint 0)	Sprint 0: Students set up complete automatic testing and building environment (Walking Skeleton)
Week 4 Project start (Sprint 1)	Demonstration of testing and building environment (Walking Skeleton) Sprint and Release Planning
Week 6 – 14 (Sprint 1 - 5) Every even week	Sprint Review & Sprint Retrospective Planning meeting of next Sprint Development of application
Week 16	Final presentation of application End of project

The topics formed a wide spectrum. There were “fun” projects like 3D multi-player games, technical challenges as well as using new technologies or frameworks.

C. Project Roles

The role of the Product Owner (PO) is very critical for the success of an agile software project, whether it is a real or academic project. The role can be filled by a student, a member of faculty, or by even someone outside the university like an industrial partner. We decided individually for each project who should take this important role. There are special challenges to master when a team decides to work with an industrial partner. The partner might have lack of knowhow or experience in agile software development. The partner sometimes also has too high expectations, which in consequence can put too much pressure on the team. The relationship with industrial partners has to be managed carefully by the lecturer. There were finally two projects with student POs, one with a member of faculty PO, and two with partners from industry. In all the projects, the Scrum Master role was taken by a student. The development teams were completely set up by students. There was the special agile coach role, which was taken by the lecturer and other faculty members.

D. User Stories and Product Backlog

After the project vision was established, the students wrote down the User Stories [17], including acceptance criteria, on paper-cards. Next, a first version of the product backlog was assembled.

E. Planning for Value

The students estimated the User Stories by playing Planning Poker [16], which usually delivers good enough results and is a lot of fun. After the User Stories were estimated and clarified, the product backlog was updated and prioritized by business value by the PO. In week four, the plan for the first iteration and a coarse-grained release plan were worked out. For the release plan, the velocity was estimated. For didactical reasons, we encouraged the students to use electronic online-tools sparingly and use pencil and paper instead.

F. Burn Down Chart and Velocity

The students drew both iteration and release burn down charts on paper. Burn down charts graphically represent work left to do versus time. The remaining work or backlog can then easily be tracked and the velocity [18] can be calculated. Once the velocity is known, the release plan can easily be updated.

G. Project setup

In week two, the team worked on an architecture prototype in order to work out which software architecture to use. This included choosing the technology stack for the realization. Table 2 gives an overview of the types and characteristics of the five projects.

TABLE II. STUDENT PROJECT SETUP

Domain	Technology	Team-size	Product Owner	Internal / External Project
3D Game	Java	10	Student	Internal
Simulation	Java/JavaScript	9	Student	Internal
Volunteering	JavaScript	8	Industry	External
Restaurant	Java/JavaScript	9	Industry	External
Accounting	Java/JavaScript	9	Faculty	Internal

TABLE III. LAB VIRTUAL MACHINE (VM)

Operating System	CentOS 6.5
Continuous Integration Server	Jenkins
Web Application Server	Glassfish
Virtualization Software	Docker [24]
Building Tools	Maven
Java Development Kit	OpenJDK 1.7
Code Versioning System (CVS)	Git [22], GitHub [23]
Database	MySQL, PostgreSQL
Monitoring	htop [25]

H. The Role of the Lecturer

As mentioned in chapter III, the lecturer takes the role of a coach or agile champion. But of course, the lecturer also represents the university. In our experience, it is important to make clear at all times, if the lecturer is coaching or representing the university.

I. Technical Setup and Practices

1) Virtual Machines

For this course, we set up a virtual machine with pre-installed software for the students to use. Each team was allocated two virtual machines: One development machine for automatic testing and Continuous Integration, and one for running the application. Table 3 describes the provided technology stack:

Each team had full access (root) to their VM so that they could install any software additionally required by their project. The students were therefore free to experiment and find their own solutions.

2) GIT and GitHub

Git [22] is a distributed version control system (VCS) designed to handle everything from small to very large projects with speed and efficiency. GitHub [23] is a web-based Git repository. GitHub provides several collaboration features such as bug tracking, feature requests, task management and wikis for every project. Git and GitHub with its collaboration features, informs the coach about the status quo of all student projects. Additionally, they keep track of the individual contributions to the code base.

3) Dependency Management with Maven

When working in an agile team with colleagues who favor different kinds of tools, tension is inevitable. Using tools like Maven [20] can ease the tension. Following the idea of *convention over configuration*, all members of the team can use their preferred development tools as long as they comply with the Maven development paradigm.

4) Continuous Integration (CI)

Continuous Integration is an absolute must to be able to deliver software with high frequency. All student teams use Maven [20] and Jenkins [21] to build and integrate their software. CI is another valuable assessment tool for the coach.

5) Automatic Testing and Code Coverage

We demand that the teams do rigorous automatic testing. Automatic testing is difficult, but in combination with refactoring it has a profound positive effect on code quality. It is important that the students have a test concept and test environment ready before they actually start coding. Therefore we introduced a “Sprint Zero” before the official project start. The goal of Sprint Zero is simple: All teams are able to do automatic testing. The students have to implement a complete test case, which demonstrates their ability to test the application from end-to-end. After every regular Sprint, code coverage is inspected and discussed with the students.

6) Clean Code and Craftsmanship

The idea of highlighting the importance of software development practice was popularized in [14]. The author introduces the disciplines, techniques, tools, and practices of software craftsmanship. Craftsmanship is much more than a technique: It is about attitude. Starting in the first semester, the students are advised to follow the Clean Code guidelines. We have observed that following and discussing these guidelines has a profound impact on the quality and design of the code base.

V. EVALUATION

A systematic evaluation of the new agile student project has not yet been executed. This evaluation will be carried out after the second round. Instead, we present our first experiences and feedback from the students. In the first round, we had a total of 45 full-time students who were split into five teams. At the time of writing, we are in the middle of the second round with 28 part-time students and three teams.

The highlight of the Software Engineering module has indisputably been the agile student project. This is true for both students and coach. The developed applications exceeded the expectations in scope, quality and student commitment. The students were initially unfamiliar with working together in large agile teams, however they soon got accustomed to it. This manifested itself in an increased productivity and creativity. We observed that the coach’s experience in agile software development is important for the outcome. Paying special attention to technical practices like automatic testing and clean code has had a positive effect on design, code quality and documentation.

Here are some written statements¹ from the students. When asked what they liked best, the students replied: “It’s great that we do Agile..”, “Practical relevance of the topics presented”, “Good coordination of lecture and project

¹ The statements are translated from German and are from the official evaluation system. All new lectures and modules of the new Computer Science study were evaluated systematically.

module”, “Best project module so far..”, “I liked that we could choose the projects ourselves..”, “Scrum!!”.

From the current part-time students we often get feedback that they can benefit from the agile practices directly in their job.

Scrum, in combination with online collaboration tools like GitHub or Jenkins CI-Server, made it possible for the coach to maintain the overview of all student projects. Coaching the projects to the desired outcome and assessing the Scrum teams was possible in an efficient and transparent manner. Evaluation and assessment of the individual student is a different story, however. Assessing individual team members is a contradiction to the agile values. Therefore, individual grades versus team grades should be considered very carefully. In the first round, all the students of a Scrum team got the same grades.

The team setup is critical with respect to finding a well-balanced team with the necessary level of knowhow, motivation and commitment. Another critical issue is the role of the Scrum Master. We let the students themselves take the role of the Scrum Master. But they usually lack the necessary experience to fulfill the requirements of that role and should therefore be specifically trained by the coach. The person who proposed the product vision typically takes the role of Product Owner. Unexpectedly, the outcome was positive with both internal and external Product Owners. However, scheduling meetings and getting feedback from external Product Owners require more effort and should be supported by the coach.

VI. CONCLUSION

In this paper we presented how educational projects can be setup in such a way that students gain experience in collaboration. The authors consider this essential since collaboration between team members, customers, users and stakeholders, is such an important part of agile software development in the industry, and because more experience in these practices, i.e. maturity, leads to increased collaboration.

The student project is best realized as a “real” agile project. The didactical concept is based on results from several industry studies, which give insight into the required competencies of agile software developers. The overall educational objective is to enable students to develop high-quality software in a functional, self-organized agile team. To achieve this the course is guided by the agile competence pyramid, which divides the competencies in three categories: Technical practices, collaboration practices and agile values. The student project aims to integrate all three by carefully setting up a realistic environment in the classroom.

The benefits of online collaboration tools like GitHub or Jenkins CI-Server make it possible for the lecturer to coach, measure progress and steer student software development projects. The authors are convinced, that software engineering education benefits significantly by embracing the modern collaboration tools the Internet has made available.

The evaluation of the course reveals very encouraging results, but also leaves some challenges and problems to be solved.

REFERENCES

- [1] Version One. State of Agile Development Survey results. <http://info.versionone.com/state-of-agile-development-survey-ninth.html>, retrieved 2.12.15
- [2] M. Kropp, A. Meier, Swiss Agile Study 2014, ISSN: 2296-2476, FHNW and ZHAW, Brugg, Zurich, www.swissagilestudy.ch
- [3] M. Kropp, A. Meier, Agile Success Factors, ISSN: 2296-2476, FHNW and ZHAW, Brugg, Zurich, www.swissagilestudy.ch
- [4] M. Sahota, An Agile Adoption and Transformation Survival Guide: Working with Organizational Culture. InfoQ. 2012.
- [5] B. Meyer, Agile!: The Good, the Hype and the Ugly, 2014, Springer, ISBN 978-3-319-05154-3
- [6] R. Biddle; M. Kropp; A. Meier, Agile Adolescence to Maturity: Experience leads to Collaboration (Workshop). OOP Conference 2016, Munich, Germany, Feb. 2016 (accepted)
- [7] V. Mahnič, Scrum in software engineering courses: an outline of the literature. Global Journal of Engineering Education 17.2 (2015).
- [8] A. Schroeder, A. Klarl, P. Mayer, Ch. Kroiß, Teaching agile software development through lab courses. Global Engineering Education Conference (EDUCON), 2012 IEEE.
- [9] A. Scharf, A. Koch, Scrum in a Software Engineering Course: An In-Depth Praxis Report, CSEE&T 2013, San Francisco, USA.
- [10] Ch. J. Stettina, Z. Zhou, Th. Bäck, B. Katzy, Academic Education of Software Engineering Practices: Towards Planning and Improving Capstone Courses Based upon Intensive Coaching and Team Routines, CSEE&T 2013, San Francisco, USA.
- [11] F. Maurer, C. Anslow, An Experience Report at Teaching a Group Based Agile Software Development Project Course. Proceedings of the 46th ACM Technical Symposium on Computer Science Education. SIGCSE '15. Kansas City, USA.
- [12] D. Rico and H. Sayani, Use of agile methods in software engineering education. In *Proceedings of the International Conference on Agile*, pages 172-179, 2009.
- [13] J. Chao, Student Project Collaboration Using Wikis, 20th Conference on Software Engineering Education & Training, 2007. CSEET '07. Dublin, Ireland.
- [14] R. C. Martin, Clean Code: A Handbook of Agile Software Craftsmanship, 2009, ISBN 0-13-235088-2
- [15] K. Schwaber, M. Beedle, Agile Software Development with Scrum, 2001, ISBN 0-13-207489-6
- [16] K. Beck, Extreme Programming Explained: Embrace Change. Addison-Wesley, 2004 ISBN 0-321-27865-8
- [17] M. Cohn, User Stories Applied, For Agile Software Development, 2004, ISBN 0-321-20568-5
- [18] M. Cohn, Agile Estimating and Planning, 2006, ISBN 0-13-147941-5
- [19] M. Kropp, A. Meier, M. Mateescu, C. Zahn, Teaching and learning agile collaboration. Proceedings 2014 IEEE, 27th Conference on Software Engineering Education and Training - (CSEE&T 2014), Klagenfurt, Austria.
- [20] Apache Maven Project, <https://maven.apache.org>, retrieved 2.12.15
- [21] Jenkins - open source continuous integration server, <https://jenkins-ci.org>, retrieved 2.12.15
- [22] Git - distributed version control system, <https://git-scm.com>, retrieved 2.12.15
- [23] GitHub - collaboration, code review, and code management <https://github.com>, retrieved 2.12.15
- [24] Docker - virtualization software, <https://www.docker.com>, retrieved 2.12.15
- [25] Htop - an interactive process viewer for Unix, <http://hisham.hm/htop/> retrieved 2.12.15